

Gato (Tic-Tac-Toe) con Minimax

Documentación Técnica

Inteligencia Artificial

Índice

1. Descripción del Proyecto	2
2. Lógica del Juego	2
2.1. Estado del tablero	2
2.2. Condición de victoria	2
3. Algoritmo Minimax	2
3.1. Concepto	2
3.2. Pseudocódigo	3
3.3. Elección del mejor movimiento	3
3.4. Análisis de complejidad	3
4. Arquitectura del Código	4
5. Flujo de Ejecución	4
6. Interfaz Gráfica	4
7. Diferencia con Negamax	4

1. Descripción del Proyecto

Este proyecto implementa el juego clásico de **Gato (Tres en Raya)** con una inteligencia artificial basada en el algoritmo **Minimax**. La IA es teóricamente imbatible: siempre gana o empata contra cualquier jugador humano.

El juego se desarrolló en **Java** con una interfaz gráfica construida con **Swing**. El tablero es de 3×3 celdas y el jugador humano compite directamente contra la CPU.

2. Lógica del Juego

2.1. Estado del tablero

El tablero se representa como un arreglo de 9 enteros donde:

- 0 = celda vacía
- 1 = jugador humano (X)
- -1 = CPU (O)

2.2. Condición de victoria

El juego verifica las 8 combinaciones ganadoras posibles: 3 filas, 3 columnas y 2 diagonales. Si alguna de estas líneas tiene los 3 valores iguales (y distintos de 0), hay un ganador.

3. Algoritmo Minimax

3.1. Concepto

Minimax es un algoritmo de búsqueda adversarial para juegos de suma cero. Construye un **árbol de juego completo** desde el estado actual hasta todos los estados terminales, y asigna una puntuación a cada nodo:

- +10: la CPU gana
- -10: el humano gana
- 0: empate

La CPU (jugador MAX) elige el movimiento de mayor puntuación; el humano (jugador MIN) elegiría el de menor puntuación.

3.2. Pseudocódigo

```
1 función minimax(tablero, profundidad, esMaximizador):  
2     si hayGanador(tablero, CPU):      retornar +10  
3     si hayGanador(tablero, humano):   retornar -10  
4     si tableroLleno(tablero):         retornar 0  
5  
6     si esMaximizador:  
7         mejorValor = -infinito
```

```

8      para cada celda vacía:
9          tablero[celda] = CPU
10         valor = minimax(tablero, prof+1, false)
11         tablero[celda] = vacío
12         mejorValor = max(mejorValor, valor)
13     retornar mejorValor
14 sino:
15     mejorValor = +infinito
16     para cada celda vacía:
17         tablero[celda] = humano
18         valor = minimax(tablero, prof+1, true)
19         tablero[celda] = vacío
20         mejorValor = min(mejorValor, valor)
21     retornar mejorValor

```

3.3. Elección del mejor movimiento

Para determinar el movimiento óptimo de la CPU:

```

1 función mejorMovimiento(tablero):
2     mejorPuntuacion = -infinito
3     mejorCelda = -1
4     para cada celda vacía:
5         tablero[celda] = CPU
6         puntuacion = minimax(tablero, 0, false)
7         tablero[celda] = vacío
8         si puntuacion > mejorPuntuacion:
9             mejorPuntuacion = puntuacion
10            mejorCelda = celda
11     retornar mejorCelda

```

3.4. Análisis de complejidad

El Gato tiene un espacio de estados máximo de $9! = 362,880$ tableros, pero con poda implícita se reduce significativamente. El árbol de juego completo tiene profundidad máxima 9 (9 celdas). La complejidad temporal es $O(b^d)$ donde b es el factor de ramificación y d la profundidad.

4. Arquitectura del Código

El proyecto se implementó en un único archivo `Gato_SG_GUI.java` que contiene:

- **Clase principal:** extiende `JFrame` y contiene la GUI completa.
- **Método `minimax()`:** implementación recursiva del algoritmo.
- **Método `mejorMovimiento()`:** evalúa todas las celdas vacías y elige la óptima.
- **Método `verificarGanador()`:** comprueba las 8 combinaciones ganadoras.
- **ActionListeners:** detectan clic del jugador y disparan la respuesta de la CPU.

5. Flujo de Ejecución

1. El jugador hace clic en una celda vacía del tablero.
2. Se registra el movimiento del humano en el arreglo de estado.
3. Se verifica si el humano ganó o si el tablero está lleno (empate).
4. Si el juego continúa, se llama a `mejorMovimiento()` que ejecuta Minimax.
5. La CPU coloca su ficha en la celda óptima.
6. Se verifica si la CPU ganó o si hay empate.
7. La interfaz se actualiza visualmente en cada paso.

6. Interfaz Gráfica

La GUI fue construida con **Java Swing**:

- `JFrame`: ventana principal del juego.
- `JBUTTON[9]`: arreglo de 9 botones que representan las celdas del tablero.
- `JLabel`: muestra el estado del juego (turno, ganador, empate).
- `GridLayout(3,3)`: organiza los 9 botones en cuadrícula.

7. Diferencia con Negamax

Minimax usa dos funciones separadas (MAX y MIN). Negamax unifica ambas aprovechando que el valor de un estado para el jugador actual es el negativo del valor para el oponente:

$$f_{\text{negamax}}(s) = -f_{\text{negamax}}(\text{sucesor}(s))$$

Ambos algoritmos producen el mismo resultado; Negamax es más compacto en código.